Task 1 | Final abstract

A retryable batch data system for municipal air-quality readings

Project purpose and users

This project designs a portable data-ingestion system for a municipality that wants to use environmental sensor readings in planning and public information services. Its operational purpose is to make historical observations consistently accessible by station and time. City planners can compare locations over a selected period; analysts can inspect pollutant and weather readings; application developers can use the stored values as input to dashboards or separate alert services. The prototype ends at reliable storage. It does not define health thresholds or send warnings, because those decisions require municipal policy, sensor-quality rules, and an accountable operating process.

Dataset and data model

The sample is UCI's Beijing Multi-Site Air Quality dataset. It contains 420,768 hourly records from 12 stations during 2013–2017. Each CSV row includes station and date/time parts, pollutant fields such as PM2.5, PM10, SO2, NO2, CO, and O3, and weather measures such as temperature, pressure, dew point, rain, wind direction, and wind speed (UCI Machine Learning Repository, n.d.). This scale is close to the half-million-record scenario and provides varied fields for modelling evolving sensors. The prototype stores one MongoDB document per station-hour. A stable outer structure contains station, timestamp, source timezone, source, and a nested measurements object. Numeric values remain numeric, missing values are omitted, and unknown measurement columns are retained. Source time is interpreted as Asia/Shanghai and stored explicitly, making the time assumption visible.

Architecture and choices

Docker Compose starts MongoDB and a Python ingestion container. The loader waits until MongoDB passes its health check, downloads the public archive, reads CSV files incrementally, transforms rows, and writes batches of 1,000. It does not load the full dataset into memory. The initialization script validates stable fields and indexes station plus timestamp.

MongoDB was chosen because a document naturally represents a reading and its flexible model can accept new sensor metrics without redesigning every record. Validation still protects the shared fields (MongoDB, n.d.-a). PostgreSQL would be suitable if the schema were fixed or joins were central; CSV alone would not provide the indexed, concurrent query service planned applications need. A larger analytical platform could later serve historical aggregates, but would add setup and cost at this stage.

Final abstract | continued

Recovery and growth

The loader derives a deterministic document ID from station and observation time. It uses unordered upserts with `$setOnInsert`, so replaying a batch after an uncertain write does not create duplicate records. Transient MongoDB errors trigger bounded exponential backoff. If a process stops between batches, an operator can run it again from the beginning: completed records remain in place and the remaining records are added. This replay strategy suits the static sample. A continuously changing source would need durable checkpoints, source versioning, a correction policy, malformed-row handling, and monitoring.

The local Compose setup demonstrates portability, but one container is not highly available. A larger deployment could use a replica set for redundancy and, if workload growth justified it, a sharded cluster. Sharding distributes data across machines but adds infrastructure complexity (MongoDB, n.d.-b). A production move would also need protected secrets, TLS, backups, access controls, monitoring, and performance tests. The shard key should be selected only after reviewing real dashboard queries; cloud hosting alone does not guarantee scalability.

Results and personal reflection

The prototype includes the loader, container definitions, database initialization script, README, and unit tests. Four unit tests pass for normalization, timezone handling, missing values, preservation of an unknown CO2 field, batching, and invalid batch size. Python compilation passes. A live Docker/MongoDB run still needs to be completed on a machine with Docker, and the GitHub URL remains a placeholder until the code is pushed.

Reflection draft to personalize: I learned that recovery depends on safe repeated writes, not only exception handling. A deterministic key and insert-only upsert address uncertain batch results; a stable outer document with flexible measurement fields addresses sensor changes. I also learned to make timestamp assumptions explicit because local sensor time and UTC differ. In similar work, I would clarify time-zone and correction rules first, test representative transformations, and verify the full deployment before adding downstream features. I still need to run the Docker smoke test and replace the repository placeholder; I should revise this reflection with my own observations from that run.

References

UCI Machine Learning Repository. (n.d.). Beijing Multi-Site Air Quality. https://archive.ics.uci.edu/dataset/501/beijing+multi+site+air+quality+data

MongoDB. (n.d.-a). Data modeling. https://www.mongodb.com/docs/manual/data-modeling/

MongoDB. (n.d.-b). Sharding. https://www.mongodb.com/docs/manual/sharding/

MongoDB. (n.d.-c). PyMongo bulk write operations. https://www.mongodb.com/docs/languages/python/pymongo-driver/current/crud/bulk-write/